E-Commerce Cart System

Files to submit: `classes.h`, `classes.cpp`, `main.cpp`

1 Overview

Implement, in C++, the order-processing core of a small e-commerce platform. One user acts as both vendor (adds products to a catalog) and customer (browses the catalog, adds items to a cart, checks out), via the fixed set of text commands in Section 6. Commands are read from a file (Section 4); invalid commands raise exceptions that are caught and logged rather than crashing the program (Section 5).

2 Class Design

Five classes: an abstract `Product` with four concrete subclasses, plus `Catalog`, `CartItem`, and `Cart`. Add constructors/destructors/helpers as needed — the members below are mandatory.

2.1 `Product` (abstract)

- `[private, static] int uniq` — next unique ID to assign, starts at 0.
- `[private] int id`, `string name`, `double basePrice`
- `[protected] double discount`, `double tax`
- `[public] double getFinalPrice()` const — Section 3.
- `[public] void view() const` — one catalog line, Section 6.

`Product` is abstract and must be stored/passed via pointer wherever a container or class needs to hold it polymorphically (this includes `Catalog`'s product list and `CartItem`'s product reference).

2.2 Subclasses

`Grocery`, `Electronics`, `Clothing`, `BabyProduct` each set `tax`/`discount` in their constructor; no other members differ from `Product`.

`add` keyword	Class	Tax	Discount
`grocery`	`Grocery`	0%	0%
`electronic`	`Electronics`	18%	12%
`clothing`	`Clothing`	18%	0%
`babyproduct`	`BabyProduct`	5%	5%

Any other keyword is invalid (Section 5).

2.3 `Catalog`

- `[private] vector<Product*> products`
- `[public] Catalog& operator+(Product* p)` — appends `p`; no output; returns `*this`.
- `[public] Product* find(int id) const` — matching product, or `nullptr`.
- `[public] void view() const` — one line per product, insertion order.

2.4 `CartItem`

- `[private] const Product* product`, `int count`, `double totalPrice`
- `[public] CartItem& operator+(int n)` — adds `n` to `count`, recomputes `totalPrice = count * product->getFinalPrice()`, returns `*this`. This is how a repeat order of a product already in the cart is merged into its existing line.
- `[public] double getTotalPrice() const`

2.5 `Cart`

- `[private] vector<CartItem> items`
- `[public] Cart& operator+(Product* p)` — adds *one unit* of `p`: merges into an existing `CartItem` for the same product via `CartItem::operator+(1)`, or else appends a new `CartItem` with count 1. Returns `*this`.
- `[public] bool order(int id, int count)` — resolves `id` to a `Product*` (`Cart` needs catalog access — give it a reference/pointer at construction, or pass the catalog in). Throws `ProductNotFoundException` if unmatched; otherwise applies `operator+` `count` times and returns `true`.
- `[public] void view() const` — one line per cart item.
- `[public] double checkout() const` — prints and returns the sum of `getTotalPrice()` over all items.

3 Pricing Rule

Store `tax`/`discount` as fractions of 1 (5% → `0.05`, not `5`).

$$\text{getFinalPrice()} = \text{basePrice} + (\text{basePrice} \times \text{tax}) - (\text{basePrice} \times \text{discount})$$

Both terms use the original `basePrice` — this is *not* a compounding calculation (do not discount then tax the discounted amount, or vice versa). Use `double` throughout; never round.

`id` is assigned from `uniq` only when a product is actually constructed, i.e. only on a successful `add`. A rejected `add` (bad category) must not consume an ID.

4 Input/Output

Commands come from a file, not `std::cin`: pass its path as `argv[1]` (`./main commands.txt`). Read with `std::ifstream` and `>>`, one character at a time (`getline` not allowed). Stop at `exit` or end of file. If the file cannot be opened, print to `std::cerr` and exit immediately (the one failure not subject to the logging below, since no command has been read yet). All normal output still goes to `std::cout`, only via `view()`/`checkout()`.

5 Exceptions and Logging

Define `ECommerceException : public std::exception` (base, with a message via `what()`), and two subclasses: `InvalidCategoryException` (thrown when an `add`'s product-type keyword is unrecognized) and `ProductNotFoundException` (thrown when an `order`'s product ID has no match). Catch both in `main.cpp` around each command's dispatch, so one bad command never stops the run and has no side effect (nothing added or changed).

At start-up open `error.log` (truncate). On each caught exception append one line:

```
ERROR: <exception message> | command: <the raw command line that failed>
```

Flush/close before exit. Standard output and the log never mix.

6 Commands

One command per line; arguments are always well-formed (only the two error cases above need handling).

- `add <type> <name> <basePrice>` — builds the product and appends it to the catalog. No output. Throws `InvalidCategoryException` on a bad `<type>`.
- `catalog` — `<name> <id> <finalPrice>` per product, insertion order.
- `order <id> <count>` — no output. Throws `ProductNotFoundException` on a bad `<id>`.
- `cart` — `<count> <id> <name> <totalPrice>` per cart line.
- `checkout` — one line: sum of `totalPrice` over all cart items.
- `exit` — terminates, no output.

7 Sample Test Cases

Sample 1

```
cart
catalog
add babyproduct seat 1000
add babyproduct cap 20
catalog
order 1 5
cart
order 0 3
```

```
cart
checkout
exit
```

Output:

```
seat 0 1000
cap 1 20
5 1 cap 100
5 1 cap 100
3 0 seat 3000
3100
```

`error.log` is empty.

Sample 2

```
add grocery basmati 368
add grocery basmati 378
add electrical LED_while_bulb 75
add grocery Mogra 448
catalog
cart
order 45 5
cart
order 2 10
order 2 10
order 10 2
cart
checkout
exit
```

Output:

```
basmati 0 368
basmati 1 378
Mogra 2 448
20 2 Mogra 8960
8960
```

`error.log`:

```
ERROR: unrecognized product category 'electrical' | command: add electrical
    LED_while_bulb 75
ERROR: no product with id 45 | command: order 45 5
ERROR: no product with id 10 | command: order 10 2
```

The two `order 2 10` lines merge into one `CartItem` (count 20) via `CartItem::operator+`; `electrical` consumes no ID, so `Mogra` still gets id 2.